A beam search for the air cargo palletization problem with weight distribution constraints

[TODO: autores]

October 23, 2026

1 Computational experiments

This section evaluates the proposed algorithm on the benchmark of Paquay et al. (2018) and on the two adapted variants introduced by Heßler et al. (2025), and compares it instance by instance with the randomized greedy search (RGS) of Heßler et al. (2025), which, to the best of our knowledge, reports the best results on these instances. Section 1.1 describes the instances, the implementation, the parameter tuning and the evaluation protocol. Section 1.2 reports the main results, Section 1.3 compares them with a lower bound, Section 1.4 analyzes the running time, Section 1.5 studies the contribution of each component and Section 1.6 the trade-off between the number of ULDs and the weight distribution.

1.1 Experimental setup

Instances. We use the 300 final instances of Paquay et al. (2018): ten groups of 30 instances with $|I| \in \{10, 20, \ldots, 100\}$ boxes sampled from a real set of 562 boxes, highly heterogeneous, with weights, allowed vertical orientations and fragile (non-stackable) boxes. Six ULD types of a Boeing 777 are available in unlimited number: three lower-deck containers (types 1–3, with up to two bottom corners cut to fit the fuselage) and three main-deck pallets and containers (types 4–6, with up to two top corners cut). Each ULD type has a weight capacity and a maximum deviation of the center of gravity (CoG) from its geometric center in the length and width directions, and a maximum height of the CoG (Paquay et al., 2016). Following Heßler et al. (2025), we consider three settings:

- **Paquay**: the original instances. A box is supported if its four bottom corners rest on non-fragile boxes or on the floor, there is no padding material, and the allowed CoG deviations are those of the instance file.
- **Adapted**: the ULD floor has an edge of width $e = 10\,\text{mm}$ for the cargo net locks, on which no box can be loaded below the height $\delta = 10\,\text{mm}$; a substructure of height δ can be used to raise the floor. A box is supported if its four bottom corners are supported or if part of its base is directly supported and at least 90% of its base is supported directly or by padding material of height at most 10 mm. The allowed CoG deviation is 10% of the half length and half width of the ULD. These are the default problem parameters of Heßler et al. (2025, Table 2).
- **One**: the adapted instances with only one ULD of each type, so that some boxes may remain unloaded. Solutions are compared lexicographically: first the number of loaded boxes, then the number of ULDs.

In the first two settings all the boxes must be loaded and the objective is to minimize the number of ULDs.

Implementation. The algorithm is implemented in C++ (compiled with g++ 13.3, `-O3`). Each run uses a single thread; the experiments were carried out on a server with two Intel Xeon E5-2620 v4 processors (8 cores each, 2.1 GHz, 3.0 GHz in turbo mode, hyper-threading disabled) and 62 GB of RAM, running 14 instances in parallel. According to the single-thread rating of PassMark (PassMark Software), one core of the Apple M2 Pro used by Heßler et al. (2025) is about 2.5 times faster than one core of our processor (4105 against 1624). The time limit t is the search time per ULD (excluding block generation); we report results for $t = 0.1\,\text{s}$ and $t = 1\,\text{s}$. The code, the instances, the solutions and the scripts that produce all the tables are available at [TODO: URL del repositorio].

Parameter tuning. The parameters of the algorithm were tuned once with irace (López-Ibáñez et al., 2016) on a separate training set and are used unchanged in the three settings and for both time limits. The training set consists of 150 preliminary instances of Paquay et al. (2018) (30 instances with 30, 40, 50, 60 and 100 boxes), disjoint from the final ones, in the Paquay setting with $t = 0.1\,\text{s}$ and a budget of 3000 runs. The cost of a run is the number of ULDs plus five times the number of ULDs whose CoG is out of range, plus $(1 - u)/2$, where u is the total utilization, to break ties in favour of smaller ULDs. Table 1 lists the tuned parameters.

Table 1: Parameters tuned with irace (training instances, Paquay setting, $t = 0.1\,\text{s}$).

Parameter	Description	Range	Value
λ (`cog_penalty`)	penalty of the CoG excess in the value of a solution	$[1, 50]$	12.66
μ (`cog_greedy`)	penalty of the CoG excess in the evaluation of a block	$[0, 5]$	0.599
`max_bl`	maximum number of blocks	$\{500, 1000, 2000, 4000\}$	500
α	exponent of the contact surface (VCS)	$[0, 5]$	1.234
β	exponent of the volume loss (VCS)	$[0, 5]$	0.239
γ	exponent of the number of boxes (VCS)	$[0, 1]$	0.224
p	tolerance of the pseudo contact surface (VCS)	$[0, 0.1]$	0.057

Validation and comparison. Every solution is written to a file (position and orientation of each box) and checked by an independent validator, which does not share code with the solver: each box is loaded at most once, in an allowed orientation, inside its ULD and its cuts, outside the floor edge below δ, without overlapping other boxes, supported according to the setting and not on top of a fragile box, and the weight capacity of each ULD is respected. The validator also counts the ULDs whose CoG is out of the allowed deviation in the length or width direction (CoG_{xy}) and above the maximum height (CoG_z, a constraint of Paquay et al. (2016) that Heßler et al. (2025) do not consider). All the 9000 solutions reported in this section are feasible. The comparison with Heßler et al. (2025) uses their instance-by-instance results (Tables 8, 10 and 12 of their online appendix). For each instance we count whether our solution is better (more boxes loaded or, with the same boxes, fewer ULDs), equal or worse, and we test the difference in the number of ULDs with the Wilcoxon signed-rank test. Each configuration is run five times; we report the mean and the standard deviation.

1.2 Results

Table 2 summarizes the results. In the Paquay setting, our algorithm uses 617.0 ULDs on average with $t = 0.1\,\text{s}$ and 597.8 with $t = 1\,\text{s}$, against 708 for the RGS of Heßler et al. (2025) and 1010 for the two-phase heuristic of Paquay et al. (2018), a reduction of 12.9% and 15.6% with respect to the RGS. It is better in 91 and 111 instances, respectively, and worse in at most one. In the adapted

setting, it uses 598.0 and 576.0 ULDs against 653 (8.4% and 11.8% fewer), and it is worse in at most one instance. In both settings the differences are highly significant (Wilcoxon signed-rank test, $p < 10^{-12}$). Moreover, almost no ULD violates the CoG constraints (at most two in 300 instances), whereas 11.2% (Paquay) and 7.2% (adapted) of the ULDs of the RGS have the CoG out of range.

In the setting with one ULD of each type, our algorithm uses 706.0 and 675.8 ULDs against 729 and leaves 22 and 21 boxes unloaded (out of 16 500) against 24, again with almost no CoG violation (3 ULDs) against 10.1% of the ULDs of the RGS. Instance by instance, it is better in 42 and 64 instances and worse in 22 and 20 ($p = 0.004$ and $p = 2 \cdot 10^{-8}$ for the number of ULDs). This setting is the closest one between the methods: with a limited number of ULDs, loading the boxes that only fit in a few ULD types matters more than the packing density, and the RGS gives the same importance to the weight balance and to the volume utilization ($I_w = 0.5$), which allows it to load boxes at the price of violating the CoG constraint (Section 1.6).

The results are almost deterministic: the five runs of each configuration differ by at most 0.5 ULDs on average, because in most ULDs the search ends before the time limit.

Table 2: Results on the 300 final instances (mean ± standard deviation over five runs). Better/worse: number of instances in which our solution is better/worse than the RGS of Heßler et al. (2025). CoG_{xy}, CoG_z: number of ULDs with the CoG out of range. Time: mean time per instance (all ULDs).

Setting	Method	ULDs	Unloaded	Better/worse	CoG_{xy}	CoG_z	Time (s)
Paquay	Paquay et al. (2018)	1010	0		12.9%	–	≤ 12
	RGS (Heßler et al., 2025)	708	0		11.2%	–	≤ 0.5
	BSG, $t = 0.1$ s	617.0 ± 0.0	0	91.0 / 0	0	0	0.62
	BSG, $t = 1$ s	597.8 ± 0.4	0	111.2 / 1	1	0	5.4
Adapted	RGS (Heßler et al., 2025)	653	0		7.2%	–	≤ 1.3
	BSG, $t = 0.1$ s	598.0 ± 0.0	0	56.0 / 1	2	0	1.2
	BSG, $t = 1$ s	576.0 ± 0.0	0	77.0 / 0	1	0	10.3
One	RGS (Heßler et al., 2025)	729	24		10.1%	–	≤ 1.1
	BSG, $t = 0.1$ s	706.0 ± 0.0	22.0 ± 0.0	42.0 / 22.0	3	0	1.6
	BSG, $t = 1$ s	675.8 ± 0.4	21.0 ± 0.0	64.4 / 20.2	2	1	13.0

[TODO: tabla por grupo de tamano ($|I| = 10, \ldots, 100$): ULDs y utilizacion mediana, como la Tabla 6 de Heßler et al. (2025); costo total y volumen total de los ULDs.]

1.3 Comparison with a lower bound

A simple lower bound on the number of ULDs is the maximum of the total volume of the boxes divided by the volume of the largest ULD, the total weight divided by the largest weight capacity, and the size of a clique of pairwise incompatible boxes (two boxes are incompatible if they cannot be placed together in the bounding box of any ULD type). In these instances the bound is always given by the volume. Our solutions reach the bound, and are therefore optimal, in 140 ($t = 0.1$ s) and 157 ($t = 1$ s) of the 300 instances of the Paquay setting, against 80 for the RGS. In the remaining instances our solutions use at most two ULDs more than the bound, but the bound is weak: it assumes that the boxes fill the largest ULD completely, whereas the non-final ULDs of our solutions are about 70% full. [TODO: cota mas fuerte o resolver las instancias pequenas con el modelo de Paquay et al. (2016).]

1.4 Running time

Heßler et al. (2025) solve each instance in 0.19 s on average (at most 0.5 s) in the Paquay setting, on an Apple M2 Pro. Figure 1 shows the number of ULDs obtained by our algorithm as a function of the mean time per instance, for time limits per ULD between 0.02 s and 2 s. With $t = 0.02$ s (0.16 s per instance on average, at most 0.52 s) our algorithm uses 638 ULDs, 9.9% fewer than the RGS, and it is worse in only one instance. Increasing the time limit keeps reducing the number of ULDs: 621 ULDs in 0.34 s, 617 in 0.62 s, 602 in 2.8 s and 596 in 10.5 s. Scaling the time of the RGS by the single-thread speed ratio of the machines (2.5), the RGS would need about 0.48 s per instance on our machine, between the mean times of our algorithm with $t = 0.05$ s (0.34 s, 621 ULDs) and $t = 0.1$ s (0.62 s, 617 ULDs): at a similar running time, our algorithm uses 12–13% fewer ULDs. This scaling is only indicative, since the two processors have different architectures.

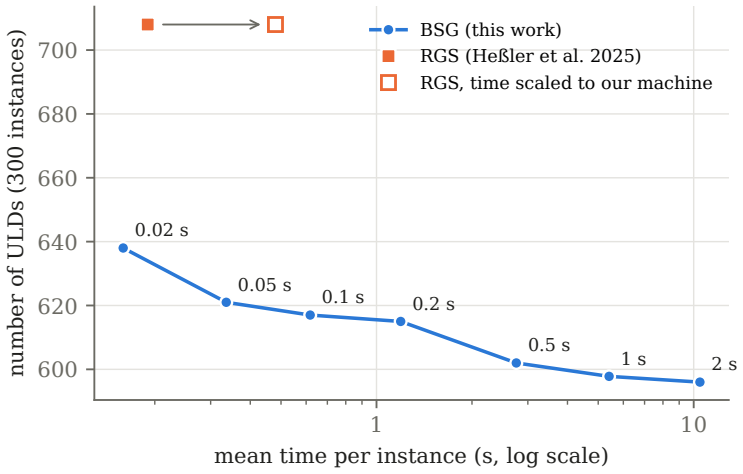


Figure 1: Number of ULDs in the 300 instances of the Paquay setting as a function of the mean time per instance (the labels are the time limits per ULD; mean of five runs for 0.1 s and 1 s, one run otherwise), and the RGS of Heßler et al. (2025): time on their machine (filled square) and scaled to ours by the single-thread speed ratio of 2.5 (empty square).

1.5 Contribution of the components

Table 3 reports the results when one component of the algorithm is removed ($t = 0.1$ s, one run). The three components that handle the CoG are essential for the weight distribution: without the penalty in the value of the solutions, the number of ULDs decreases by 2–27 but 193–353 ULDs have the CoG out of range; without the translation of the load the CoG is out of range in 84–158 ULDs, and without the grouped reload in 20–31 ULDs, which also increases the number of ULDs (by 5–20) and, with one ULD of each type, the number of unloaded boxes (from 22 to 39). The penalty of the CoG in the evaluation of the blocks saves 7–9 ULDs without increasing the violations, the elimination of ULDs saves 1–13 ULDs, and the substructure saves 20–25 ULDs in the adapted settings. Removing all of them except the penalty and the translation increases the number of ULDs by 13 (Paquay), 36 (adapted) and 98 (one).

Table 3: Ablation ($t = 0.1$ s, one run): number of ULDs and number of ULDs with the CoG out of range in the length or width direction (in the setting *One*, also the number of unloaded boxes).

	Paquay		Adapted		One	
Without	ULDs	CoG_{xy}	ULDs	CoG_{xy}	ULDs (unloaded)	CoG_{xy}
– (complete algorithm)	617	0	598	2	706 (22)	3
CoG in the evaluation of the blocks	625	2	607	6	713 (29)	2
CoG penalty in the value	615	193	590	296	679 (17)	353
translation of the load	624	84	602	84	723 (28)	158
grouped reload	622	25	604	31	726 (39)	20
elimination of ULDs	620	0	599	2	719 (22)	4
substructure	–	–	618	1	731 (28)	6
all except penalty and translation	630	26	634	61	804 (39)	54

1.6 Number of ULDs versus weight distribution

The two methods treat the CoG differently: Heßler et al. (2025) combine the volume utilization and the weight balance in the score of a solution with weights $1 - I_w$ and I_w, with $I_w = 0.5$ in the adapted settings, whereas our penalty λ makes the CoG almost a hard constraint. As the ablation shows, relaxing it reduces the number of ULDs and of unloaded boxes at the price of many CoG violations. To compare the methods independently of this choice, Figure 2 shows the number of ULDs against the percentage of ULDs with the CoG out of range for $\lambda \in \{0, 0.5, 1, 2, 5, 12.66, 30\}$ ($t = 0.1$ s). In the Paquay and adapted settings the number of ULDs is almost insensitive to λ as long as $\lambda \geq 0.5$ (617–620 and 593–599 ULDs, with at most 9 ULDs out of range); only $\lambda = 0$, which ignores the CoG in the value of the solutions, saves a few more ULDs (616 and 591) with 31% and 49% of the ULDs out of range. In both settings every value $\lambda \geq 0.5$ dominates the RGS, which uses more ULDs and has more violations. In the setting with one ULD of each type, every value $\lambda \geq 5$, including the tuned one, dominates the RGS in the three criteria (705–710 ULDs, 22–23 unloaded boxes and 0.4% of the ULDs out of range, against 729, 24 and 10.1%); with $\lambda = 0.5$ the algorithm uses 699 ULDs and leaves 20 boxes unloaded with 2.7% of the ULDs out of range, and with $\lambda = 0$ it uses 679 ULDs and leaves 19 boxes unloaded, but half of the ULDs violate the CoG constraint.

References

Katrin Heßler, Timo Hintsch, and Lukas Wienkamp. A fast optimization approach for a complex real-life 3D Multiple Bin Size Bin Packing Problem. *European Journal of Operational Research*, 2025. arXiv:2410.01445.

Manuel López-Ibáñez, Jérémie Dubois-Lacoste, Leslie Pérez Cáceres, Mauro Birattari, and Thomas Stützle. The irace package: Iterated racing for automatic algorithm configuration. *Operations Research Perspectives*, 3:43–58, 2016.

Céline Paquay, Michael Schyns, and Sabine Limbourg. A mixed integer programming formulation for the three-dimensional bin packing problem deriving from an air cargo application. *International Transactions in Operational Research*, 23(1-2):187–213, 2016.

Céline Paquay, Sabine Limbourg, and Michael Schyns. A tailored two-phase constructive heuristic for the three-dimensional Multiple Bin Size Bin Packing Problem with transportation constraints. *European Journal of Operational Research*, 267(1):52–64, 2018.

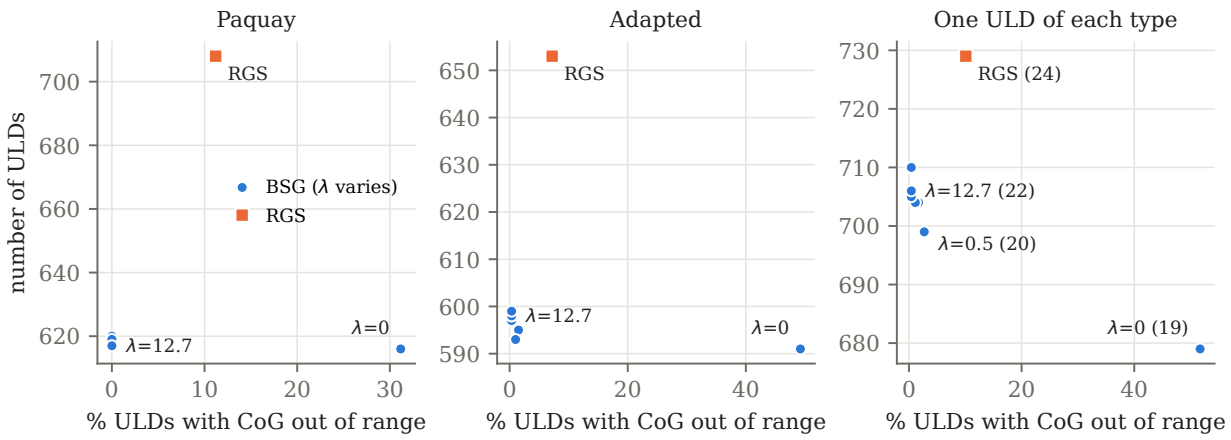


Figure 2: Number of ULDs against the percentage of ULDs with the CoG out of range in the length or width direction, for several values of the penalty λ ($t = 0.1\,\mathrm{s}$), and the RGS of Heßler et al. (2025). In the setting with one ULD of each type the number of unloaded boxes is given in parentheses.

PassMark Software. CPU Benchmarks: single thread performance. https://www.cpubenchmark.net. Accessed October 2026.